📘 DEPLOYMENT OVERVIEW | Gist on How we are going to do CAP2 Deployment

🔹 Goal

Make the ChainCert system accessible to users for UAT by hosting it on a cloud server.

🔹 Core Idea

Take your current **local setup** and **replicate it inside a cloud VM (AWS EC2)**, then expose it to the internet.

🔹 High-Level Flow

1. Launch EC2 instance (Ubuntu) (yes magbabayad tayo)
2. SSH into EC2
3. Clone project from GitHub para makuha malagay mga files sa ec2 instance
4. Instead of docker desktop, we use docker terminal, yung mano mano
5. RUN SETUP (same as local) → Kung ano man setup/commands we did in ubuntu terminal, ganun lang rin sa ec2 instance para ma:
    - Start Fabric network
    - Setup CouchDB
    - Enroll identities and so on
6. Open ports in AWS Security Group para maaccess ng madla
7. Access system via: http://<EC2-PUBLIC-IP>:3000
8. OPTIONALLY, we also deploy using **VERCEL, basically para mas malinis public url, from** http://<EC2-PUBLIC-IP>:3000 TO https/chaincert.com chuchu

---

📘 DEPLOYMENT FAQs

❓ What is AWS EC2 and why do we need it?

Tropang Explanation/Summary of Answer

👉 EC2 is your **server in the cloud** that runs your system 24/7 even if your laptop is off, to make it possible to do UAT/users to access site kahit nakaoff lappy mo

---

In depth/technical answer:

AWS EC2 is a virtual machine that provides:

- its own OS (Ubuntu)
- CPU, RAM, storage
- network access

Unlike your local machine:

- it stays online continuously
- it is publicly accessible

This solves the main deployment problem:

Local system = only works when your laptop is ON
EC2 system = always accessible for UAT and demo

Analogy/Example/Scenario

Your laptop = temporary workstation
EC2 = rented shop that stays open all day

❓ Is EC2 like a laptop with GUI?

Tropang Explanation/Summary of Answer

👉 No — it's a **server**, not a desktop. Parang ubuntu terminal basically.

In depth/technical answer:

EC2 instances:

- usually run **headless Ubuntu (no GUI)**
- are accessed via SSH terminal

You don't use:

- Chrome
- Docker Desktop

Instead, you use:

- CLI tools (`docker`, `node`, etc.)

Analogy/Example/Scenario

Laptop = personal computer with screen

EC2 = factory machine you control remotely

❓ How do we move our files to EC2?

Tropang Explanation/Summary of Answer

👉 Use **Git clone (best)** or **SCP upload**.

In depth/technical answer:

Option 1 (Recommended):

git clone <your-repo>

Option 2:

scp -r <local-folder> ubuntu@<EC2-IP>:~

Do NOT use:

/mnt/c/...

because that only works in WSL.

Analogy/Example/Scenario

EC2 = empty apartment

Git/SCP = moving your furniture in

❓ Do we copy Fabric files like TLS certs and MSP?

Tropang Explanation/Summary of Answer

👉 No — you **recreate them** in EC2.

In depth/technical answer:

Do NOT copy:

- `organizations/`
- TLS certs
- MSP folders
- wallets

Instead:

- rerun setup scripts
- regenerate identities

Reason:

- these are environment-specific
- copying them causes errors

Analogy/Example/Scenario

Don’t bring cooked food to a new kitchen

Bring the recipe and cook again

❓ Will Docker still work without Docker Desktop?

Tropang Explanation/Summary of Answer

👉 Yes — Docker CLI is the **standard in servers**.

In depth/technical answer:

In EC2:

- install Docker Engine
- use commands like:

docker ps

docker compose up -d

Docker Desktop is just a GUI wrapper.

Analogy/Example/Scenario

Docker Desktop = remote control

Docker CLI = direct control panel

❓ Can we run backend and frontend at the same time?

Tropang Explanation/Summary of Answer

👉 Yes — use **multiple terminals or background processes**.

In depth/technical answer:

Options:

- multiple SSH sessions
- `tmux`
- `pm2` (recommended)

Example:

node sdk-server-fixed.js

npm start

Better:

pm2 start sdk-server-fixed.js

Analogy/Example/Scenario

One machine, multiple workers doing different tasks

❓ How do we make the site accessible to users?

Tropang Explanation/Summary of Answer

👉 Open ports in AWS + run frontend → access via public IP.

In depth/technical answer:

1. Open ports in Security Group:
    - 3000 (frontend)
    - 4000 (backend)
2. Run frontend
3. Access:

http://<EC2-IP>:3000

Analogy/Example/Scenario

Opening ports = unlocking your shop door

Public IP = your shop address

❓ Why is my frontend not working after deployment?

Tropang Explanation/Summary of Answer

👉 Most likely using `localhost` instead of EC2 IP.

In depth/technical answer:

Wrong:

localhost:4000

Correct:

http://<EC2-IP>:4000

Because:

- browser runs on user machine
- not on EC2

Analogy/Example/Scenario

Telling customers to go to “your house” instead of your shop

❓ What is Vercel and should we use it?

Tropang Explanation/Summary of Answer

👉 Vercel is for **frontend hosting only** — optional.

In depth/technical answer:

Vercel:

- hosts React frontend
- gives public URL

Still need EC2 for:

- backend
- Fabric
- CouchDB

Analogy/Example/Scenario

Vercel = storefront

EC2 = kitchen

❓ Do we need Vercel for deployment?

Tropang Explanation/Summary of Answer

👉 No — EC2 alone is enough.

In depth/technical answer:

EC2 can host:

- frontend
- backend
- blockchain

Vercel is only for:

- better UI hosting
- cleaner URL

Analogy/Example/Scenario

You don’t need a fancy sign to run a restaurant

❓ What is the final deployment architecture?

Tropang Explanation/Summary of Answer

👉 One EC2 server running everything.

In depth/technical answer:

Inside EC2:

- Docker → Fabric + CouchDB

- Node.js → backend
- React → frontend

External:

- users access via browser

Analogy/Example/Scenario

One building:

- kitchen (backend + Fabric)
- storage (CouchDB)
- storefront (frontend)

Alternative for AWS EC2

Digital Ocean

https://www.digitalocean.com/

Will look into it pa (cheaper and easier to setup than AWS daw)

Why DigitalOcean is Easier (VERY BASIC)

🧠 1. Fewer Steps to Start

👉 **DigitalOcean:**

- Click “Create Droplet”
- Choose Ubuntu
- Done ✅

👉 **AWS EC2:**

- Choose instance
- Create key pair
- Configure VPC
- Configure security group
- Review settings

❗ More steps = more chances to mess up

🎛️ 2. Cleaner Interface (Less Confusing)

👉 DigitalOcean:

- Simple dashboard
- Only shows what you need

👉 AWS:

- So many menus (EC2, VPC, IAM, Security, etc.)
- Easy to get lost

🔐 3. Less Configuration Needed

👉 DigitalOcean:

- Server is ready to use immediately

👉 AWS:

- You must manually handle:
    - Security groups (firewall)
    - Permissions
    - Network settings

🚪 4. Ports Are Simpler

👉 DigitalOcean:

- Usually open already or easy to configure

👉 AWS:

- You must **explicitly open ports** (3000, 4000)
- If you forget → your app won't work

💸 5. No Billing Confusion

👉 DigitalOcean:

- Fixed price (e.g., $5/month)

👉 AWS:

- Pay for:
    - CPU usage
    - Storage
    - Data transfer

❗ Easy to get unexpected charges

🐳 6. Better for Your Use Case (Docker + Fabric)

👉 DigitalOcean:

- Works like a normal Linux computer
- No restrictions

👉 AWS:

- More complex environment
- Sometimes extra setup needed

🏠 Simple Analogy

👉 DigitalOcean = plug in and use
👉 AWS = build everything before using

🏁 One-line Explanation (for reporting)

👉 “DigitalOcean is simpler than AWS EC2 because it requires fewer setup steps, has a more user-friendly interface, and involves less configuration, making it ideal for quick deployment of projects.”

If you want:
👉 I can give you a **defense-ready script (like how to explain this to your prof)**
👉 or a **side-by-side demo flow (EC2 vs DigitalOcean steps)**